AI Engineering Project

Multi-Step Hourly Traffic Volume Forecasting

Using Machine Learning and Deep Learning

AI Engineering Project Team

6 Members — 1 Mentor

Addis Ababa, Ethiopia

August 2026

Abstract

This project investigates multi-step hourly traffic volume forecasting using the Metro Interstate Traffic Volume dataset collected from Interstate 94. The objective is to predict future traffic volume at 1-, 3-, 6-, 12-, and 24-hour horizons while preserving the temporal structure of the data and avoiding information leakage. Four forecasting approaches were evaluated: a Seasonal Naive baseline, Prophet with external regressors, XGBoost with engineered time-series features, and a Long Short-Term Memory (LSTM) neural network. The models were evaluated using Mean Absolute Error (MAE), Symmetric Mean Absolute Percentage Error (sMAPE), and Mean Absolute Scaled Error (MASE). Experimental results show that XGBoost provides the best performance for short-term horizons, achieving MASE values of 0.5197, 0.6390, and 0.7280 at 1, 3, and 6 hours, respectively. LSTM performs best at the 12-hour horizon with a MASE of 0.8105, while the Seasonal Naive baseline achieves the lowest error at 24 hours with a MASE of 0.9404. These findings demonstrate that forecasting performance varies substantially with the prediction horizon and that a simple seasonal baseline can remain highly competitive for longer-term forecasting.

Contents

1 Introduction **4**

2 Problem Statement **4**

3 Objectives **4**

4 Dataset Description **5**

5 Methodology **5**
5.1 Data Quality and Preprocessing 5
5.2 Exploratory Data Analysis 6
5.3 Feature Engineering 8
5.4 Forecasting Strategy 8

6 Forecasting Models **8**
6.1 Seasonal Naive 9
6.2 Prophet 9
6.3 XGBoost 9
6.4 Long Short-Term Memory (LSTM) 10

7 Experimental Setup **11**
7.1 Train/Validation/Test Split 11
7.2 Cross-Validation 11
7.3 Evaluation Metrics 11

8 Experimental Results **12**
8.1 Overall Model Comparison 12
8.2 Best Model Summary 12
8.3 MASE Comparison 13
8.4 MAE Comparison 13
8.5 Actual vs Predicted 14
8.6 XGBoost Feature Importance 14

9 Discussion **15**
9.1 Key Findings 15
9.2 Best Model by Horizon 15

10 Limitations **15**

11 Conclusion **16**

12 Future Work **16**

13 References **17**

1 Introduction

Traffic forecasting is critical for urban planning, congestion management, and routing optimization. Hourly forecasting is particularly challenging due to high variance, complex daily and weekly seasonalities, and the impact of exogenous variables like weather. This documentation outlines the methodology and findings of a multi-step forecasting prototype developed by our unified engineering project team. Time-series forecasting differs from ordinary machine learning because temporal order must be strictly preserved; standard random cross-validation cannot be used. Avoiding future-data leakage is paramount, requiring rigorous chronological splitting and careful feature engineering to ensure models do not "see" the future during training.

2 Problem Statement

The objective of this project is to build a robust multi-step time-series forecasting system to predict hourly traffic volume. Specifically, the system must generate independent predictions for the following future horizons:

- 1-hour ahead
- 3-hour ahead
- 6-hour ahead
- 12-hour ahead
- 24-hour ahead

3 Objectives

- Preprocess historical traffic data, handling duplicate timestamps and large temporal gaps.
- Analyze temporal patterns through exploratory data analysis (EDA).
- Engineer robust time-series features without data leakage.
- Build and tune statistical, machine learning, and deep learning forecasting models.
- Implement expanding-window time-based cross-validation to simulate real-world model deployment.
- Identify the most effective forecasting approach across different time horizons.

4 Dataset Description

The project utilizes historical traffic data from Interstate 94, combined with weather and holiday indicators.

Property	Description
Dataset	Metro Interstate Traffic Volume
Target Variable	`traffic_volume`
Frequency	Hourly
Location	I-94
Period	2012-10-02 to 2018-09-30
Observations (Raw)	48,204
Cleaned Observations	40,575
Exogenous Variables	`holiday`, `temp`, `rain_1h`, `snow_1h`, `clouds_all`, `weather_main`

Table 1: Dataset Summary

5 Methodology

To ensure rigorous evaluation and prevent data leakage, the project followed a strict chronological pipeline.

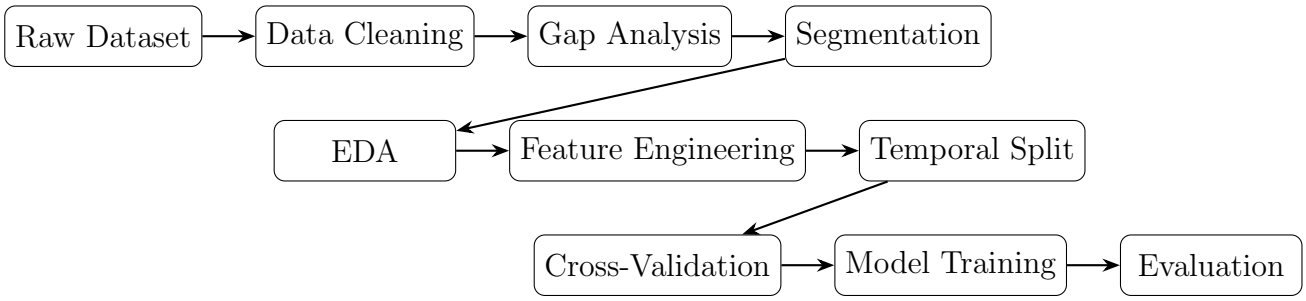

Figure 1: Overall Traffic Forecasting Methodology Pipeline.

5.1 Data Quality and Preprocessing

Strict data sanitization was applied to ensure continuous, reliable time-series modeling. The `date_time` column was converted to standard datetime objects, and the dataset was sorted chronologically. The raw dataset contained 5,445 duplicate timestamp groups. Analysis confirmed that the target variable (`traffic_volume`) was identical across all duplicates.

After removing duplicate records, 40,575 unique observations remained. The subsequent gap analysis was performed against the theoretical continuous hourly timeline, revealing 11,976 missing timestamps. The largest continuous gap was 7,386 hours (approximately 307.8 days). To prevent features from spanning across large data gaps, a gap threshold of >24 hours was established, splitting the data into 13 isolated temporal segments.

5.2 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was implemented to visualize traffic trends and seasonal constraints.

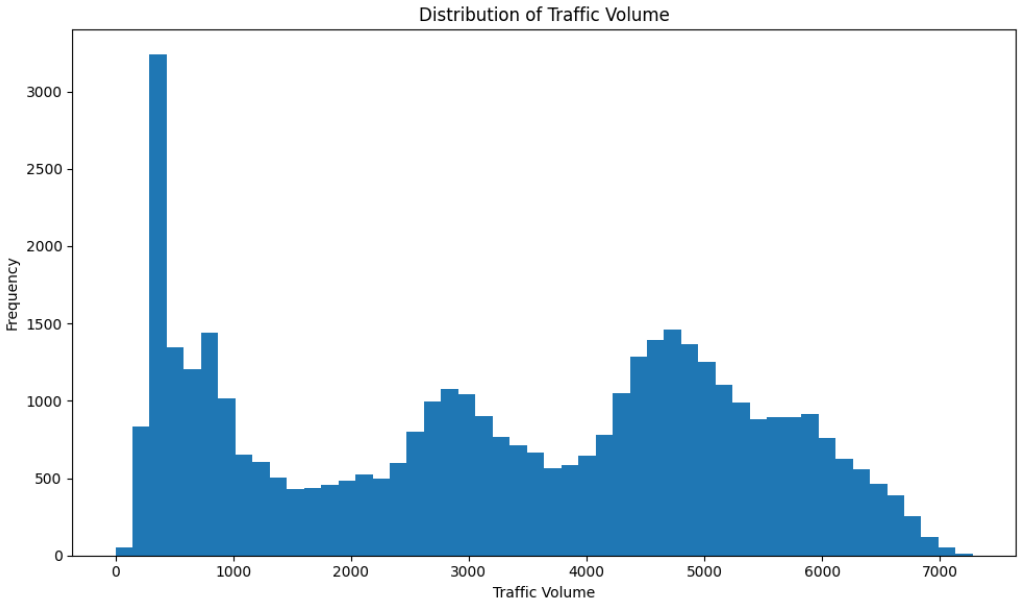

Figure 2: Distribution of Traffic Volume.

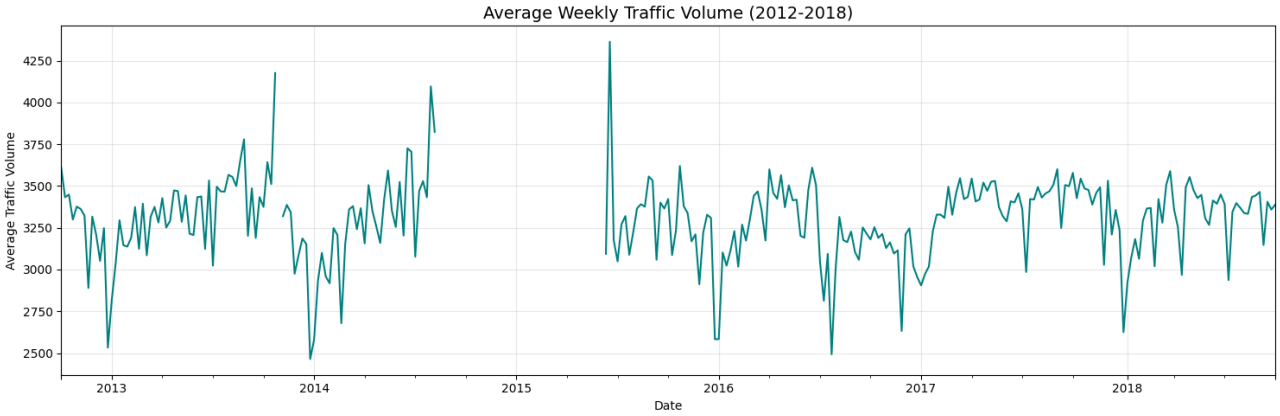

Figure 3: Average Weekly Traffic Volume (2012-2018).

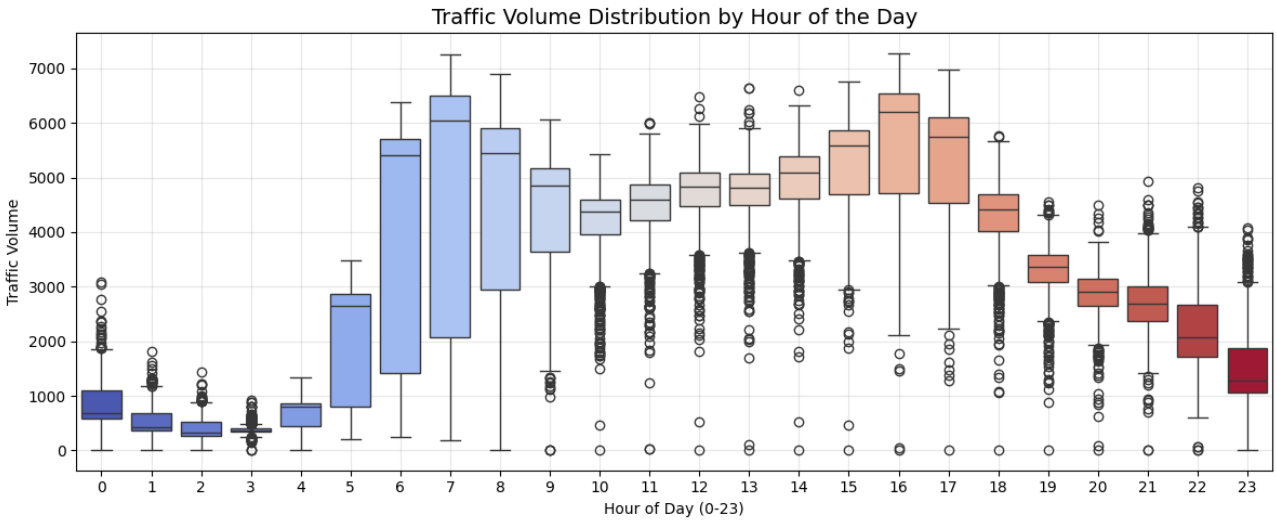

Figure 4: Traffic Volume Distribution by Hour of the Day.

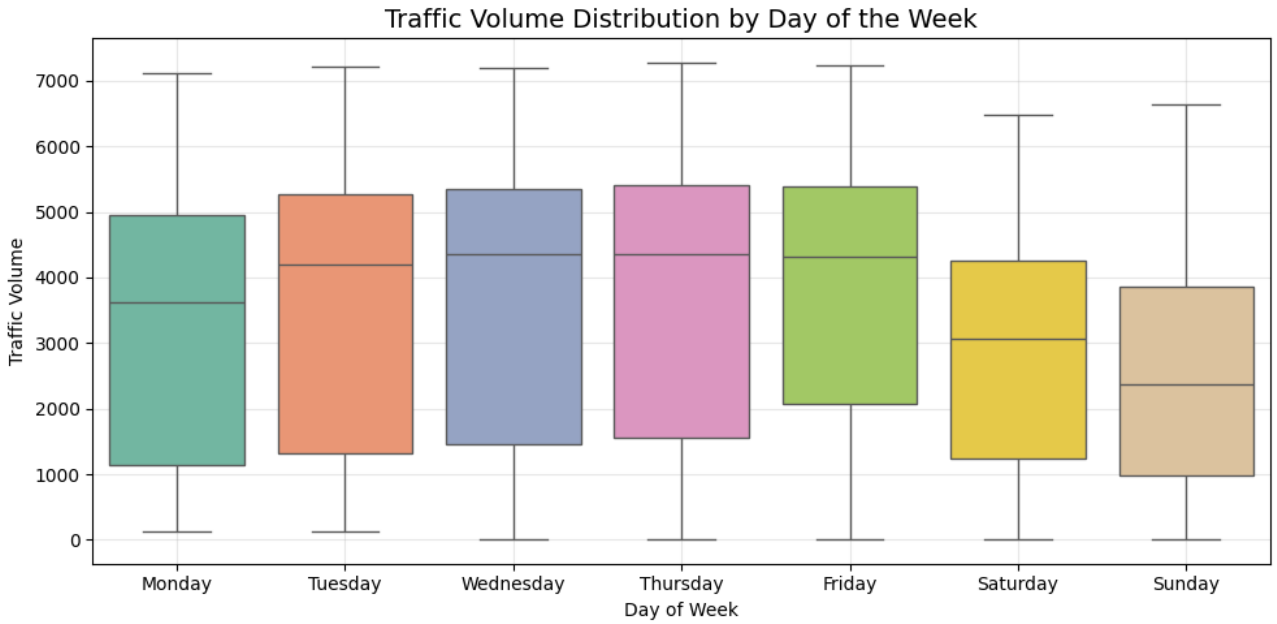

Figure 5: Traffic Volume Distribution by Day of the Week.

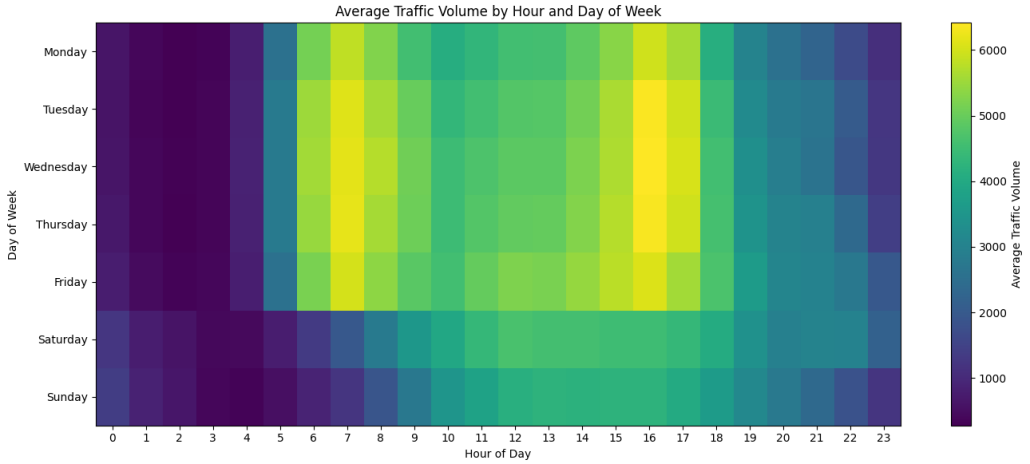

Figure 6: Average Traffic Volume by Hour and Day of Week.

5.3 Feature Engineering

- **Lag Features:** `shift(1)`, `shift(24)`, and `shift(168)` were computed within segments to capture explicit auto-regressive relationships.
- **Rolling Features:** 3-hour and 24-hour rolling means and standard deviations were calculated after shifting the target by 1 hour to prevent leakage.
- **Calendar Features:** `hour`, `day_of_week`, `month`, and `is_weekend`. Cyclical encoding (sine/cosine) was applied to `hour` and `day_of_week`.

5.4 Forecasting Strategy

The project utilizes a **direct forecasting strategy**. A distinct target variable is created for each specific horizon by shifting the volume negatively (`shift(-h)`). A separate model is trained for each horizon, directly mapping current inputs to the future target.

6 Forecasting Models

Model	Type	Main Inputs	Strength
Seasonal Naive	Baseline	Historical seasonal value	Simple, highly robust to noise
Prophet	Statistical	Time + regressors	Interprets trend/seasonality
XGBoost	Machine Learning	Lag + rolling + calendar	Captures nonlinear relationships
LSTM	Deep Learning	Sequential observations	Learns temporal dependencies

Table 2: Summary of Evaluated Models

6.1 Seasonal Naive

The Seasonal Naive model was used as the primary baseline for evaluating the performance of the more advanced forecasting models. Rather than learning complex relationships from the data, the model assumes that the traffic volume at a future time will be similar to the traffic volume observed during the corresponding seasonal period.

Since traffic exhibits strong weekly seasonality, a seasonal period of 168 hours (24 hours $\times$ 7 days) was selected. Therefore, the prediction for a future horizon h is obtained from the observation located 168 hours before the target time:

$$\hat{y}_{t+h} = y_{t+h-168} \tag{1}$$

For example, for a 3-hour-ahead forecast:

$$\hat{y}_{t+3} = y_{t+3-168} = y_{t-165} \tag{2}$$

Similarly, for a 24-hour-ahead forecast:

$$\hat{y}_{t+24} = y_{t+24-168} = y_{t-144} \tag{3}$$

Although this approach is simple and requires no model training, it provides an important benchmark. A forecasting model should demonstrate that its additional complexity provides a meaningful improvement over this seasonal reference.

6.2 Prophet

Prophet is a time-series forecasting model based on an additive regression framework. It is designed to represent important components of a time series, including trend, seasonality, and external effects.

In this project, Prophet was extended with external regressors using the `add_regressor` functionality. Weather and calendar variables were incorporated to provide additional information that could influence traffic volume. The included regressors consisted of temperature, rainfall, snowfall, cloud coverage, weekend indicators, and holiday indicators.

Prophet was evaluated separately for each forecasting horizon using the same chronological evaluation framework as the other models. This allowed its performance to be compared fairly against the Seasonal Naive, XGBoost, and LSTM approaches.

6.3 XGBoost

Extreme Gradient Boosting (XGBoost) is a supervised machine learning algorithm based on an ensemble of decision trees. The model builds trees sequentially, with each new tree attempting to correct errors made by the previous trees. This allows XGBoost to model complex and nonlinear relationships between historical traffic patterns and the target variable.

For this project, XGBoost was provided with engineered time-series features, including lag variables, rolling statistics, calendar variables, and cyclical time representations. These features allow the model to capture both short-term traffic momentum and recurring daily and weekly patterns.

The main model configuration used in the experiments was:

- **Number of estimators:** 500
- **Maximum tree depth:** 8
- **Learning rate:** 0.05

A separate XGBoost model was trained for each forecasting horizon. This direct forecasting approach allowed the model to learn a horizon-specific relationship between the available historical information and the future traffic volume.

6.4 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) designed to learn dependencies in sequential data. Unlike conventional neural networks, LSTM networks maintain an internal memory state that allows information from earlier observations to influence later predictions.

This property makes LSTM suitable for traffic forecasting because traffic volume contains temporal dependencies and recurring patterns that may extend across many hours. In this project, the LSTM model used a sequence length of 168 hours, corresponding to one complete week of historical observations. This provided the network with sufficient historical context to learn daily and weekly traffic patterns.

The implemented architecture consisted of an LSTM layer followed by a Dropout layer and a Dense output layer:

- **Sequence Length:** 168 hours
- **Architecture:** LSTM $\rightarrow$ Dropout $\rightarrow$ Dense
- **Loss Function:** Mean Squared Error (MSE)
- **Optimizer:** Adam

The **Adam optimizer** (Adaptive Moment Estimation) is an optimization algorithm commonly used for training neural networks. It updates the network's weights during training based on the gradients of the loss function. Adam combines ideas from momentum-based optimization and adaptive learning rates, allowing different model parameters to receive appropriately scaled updates. This generally provides stable and efficient training for neural networks such as LSTM.

The Mean Squared Error (MSE) loss function was used to measure the difference between predicted and actual traffic volumes during training. The Adam optimizer then used the gradients of this loss to iteratively adjust the network weights and improve its predictions.

7 Experimental Setup

7.1 Train/Validation/Test Split

To strictly preserve temporal order, the final 15% of chronologically ordered observations were reserved as the unseen test set.

7.2 Cross-Validation

To assess model stability before final testing, expanding-window cross-validation was performed using three chronological validation folds. Unlike conventional random cross-validation, each validation fold occurs after its corresponding training period, preserving the temporal structure of the dataset.

The cross-validation results were used for model comparison and model selection, while the final performance reported in the Experimental Results section was obtained from the unseen chronological test set. This separation ensured that the test set remained independent of the model development process.

7.3 Evaluation Metrics

Models were evaluated using the following three metrics. Lower values indicate superior performance for all three measures:

- **Mean Absolute Error (MAE):** Measures the average magnitude of absolute errors in vehicles per hour.
- **Symmetric Mean Absolute Percentage Error (sMAPE):** Provides a scale-independent percentage evaluation of the forecast error.
- **Mean Absolute Scaled Error (MASE):** Scales the error relative to the in-sample performance of the seasonal naive baseline.

8 Experimental Results

8.1 Overall Model Comparison

The table below displays the numerical performance of all models on the chronological holdout test set.

Model	Horizon (h)	MAE	sMAPE (%)	MASE
Seasonal Naive	1	318.6117	11.6769	0.9402
Prophet	1	612.0733	32.6822	1.8061
XGBoost	1	176.1365	7.4493	0.5197
LSTM	1	356.7979	22.2246	1.0528
Seasonal Naive	3	318.4985	11.6830	0.9398
Prophet	3	612.1979	32.6917	1.8065
XGBoost	3	216.5343	9.4532	0.6390
LSTM	3	325.5380	13.3323	0.9606
Seasonal Naive	6	318.1696	11.6833	0.9389
Prophet	6	612.3313	32.7038	1.8069
XGBoost	6	246.7107	11.2880	0.7280
LSTM	6	289.5374	11.4872	0.8544
Seasonal Naive	12	318.1722	11.6845	0.9389
Prophet	12	612.5248	32.6926	1.8075
XGBoost	12	295.1253	13.9565	0.8709
LSTM	**12**	**274.6781**	**12.1764**	**0.8105**
Seasonal Naive	**24**	**318.6929**	**11.6681**	**0.9404**
Prophet	24	611.8561	32.6236	1.8055
XGBoost	24	351.7814	16.6423	1.0380
LSTM	24	361.5113	14.9656	1.0668

Table 3: Final Test Set Metric Comparison Across All Models and Horizons

8.2 Best Model Summary

To easily identify the optimal forecasting strategy, the best-performing model for each distinct horizon is summarized below.

Horizon	Best Model	MAE	MASE
1 hour	XGBoost	176.1365	0.5197
3 hours	XGBoost	216.5343	0.6390
6 hours	XGBoost	246.7107	0.7280
12 hours	LSTM	274.6781	0.8105
24 hours	Seasonal Naive	318.6929	0.9404

Table 4: Best-performing model at each forecasting horizon.

8.3 MASE Comparison

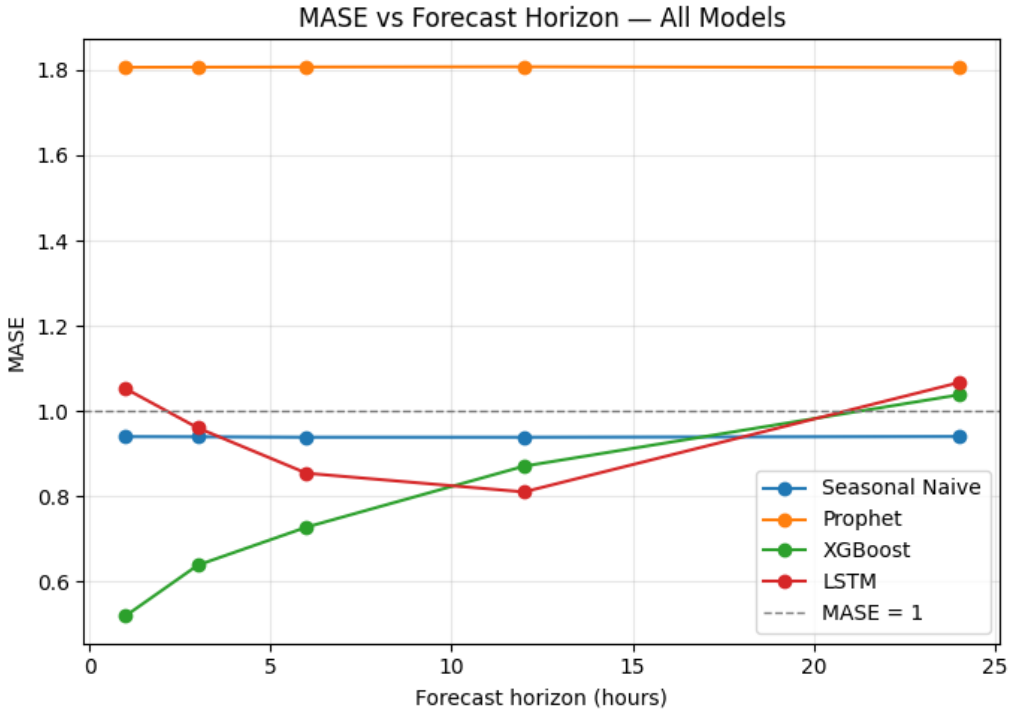

Figure 7: MASE across forecasting horizons for all evaluated models.

8.4 MAE Comparison

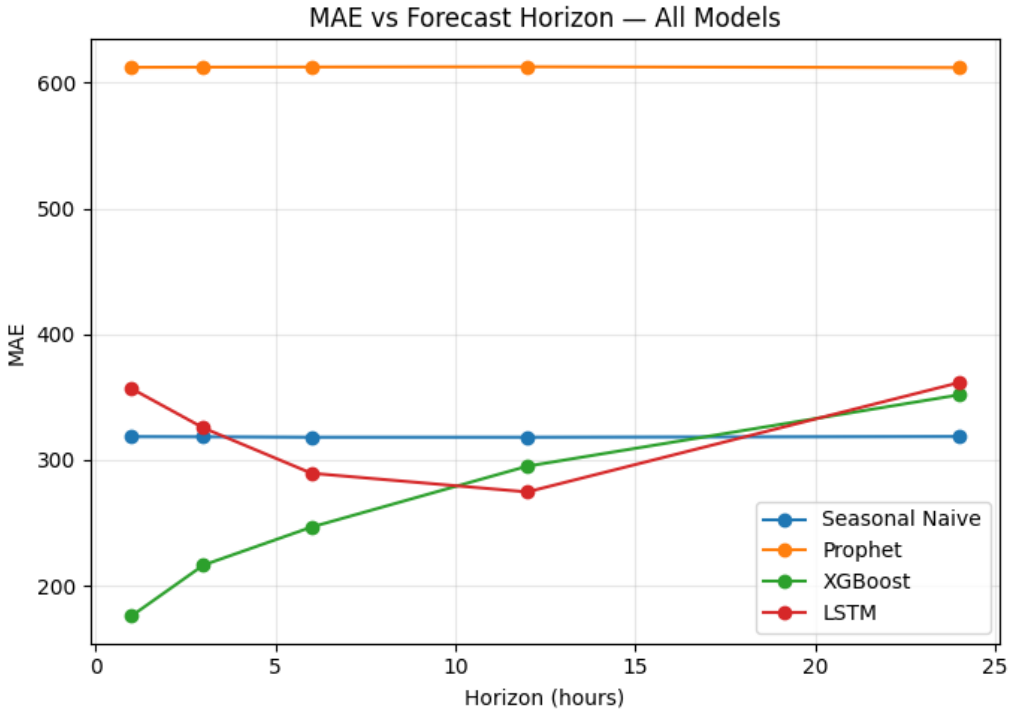

Figure 8: MAE across forecasting horizons for all evaluated models.

8.5 Actual vs Predicted

Figure 9 compares the observed traffic volume with the predictions generated by XGBoost for the 1-hour forecasting horizon. The predicted values closely follow the general temporal pattern of the observed series, particularly during periods of increasing and decreasing traffic volume. The relatively close alignment between the two series is consistent with the low MAE and MASE obtained by XGBoost at this horizon.

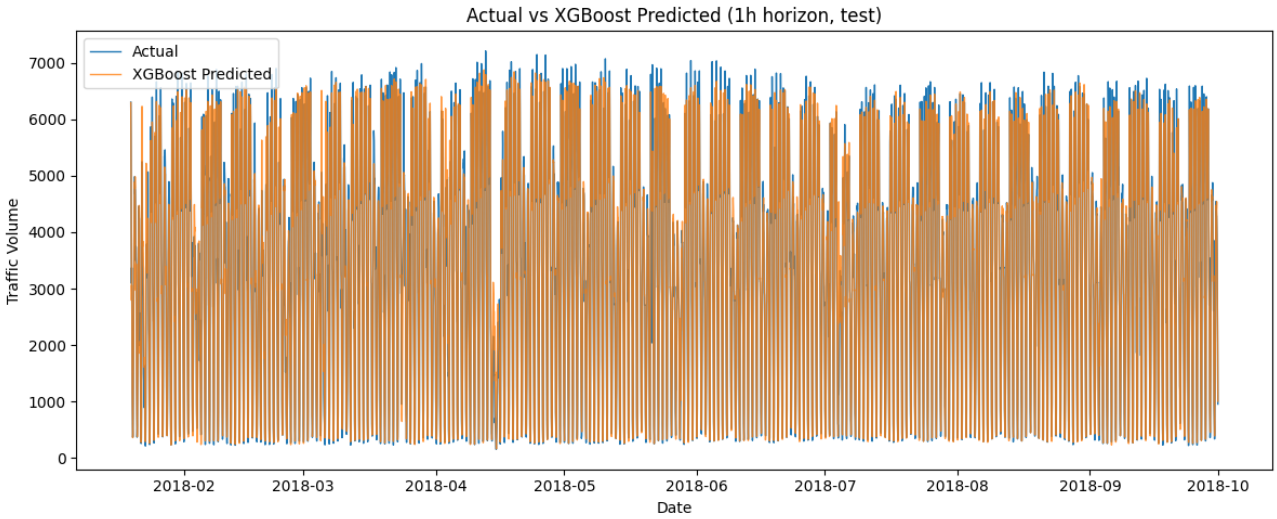

Figure 9: Actual versus XGBoost-predicted traffic volume for the 1-hour forecasting horizon.

8.6 XGBoost Feature Importance

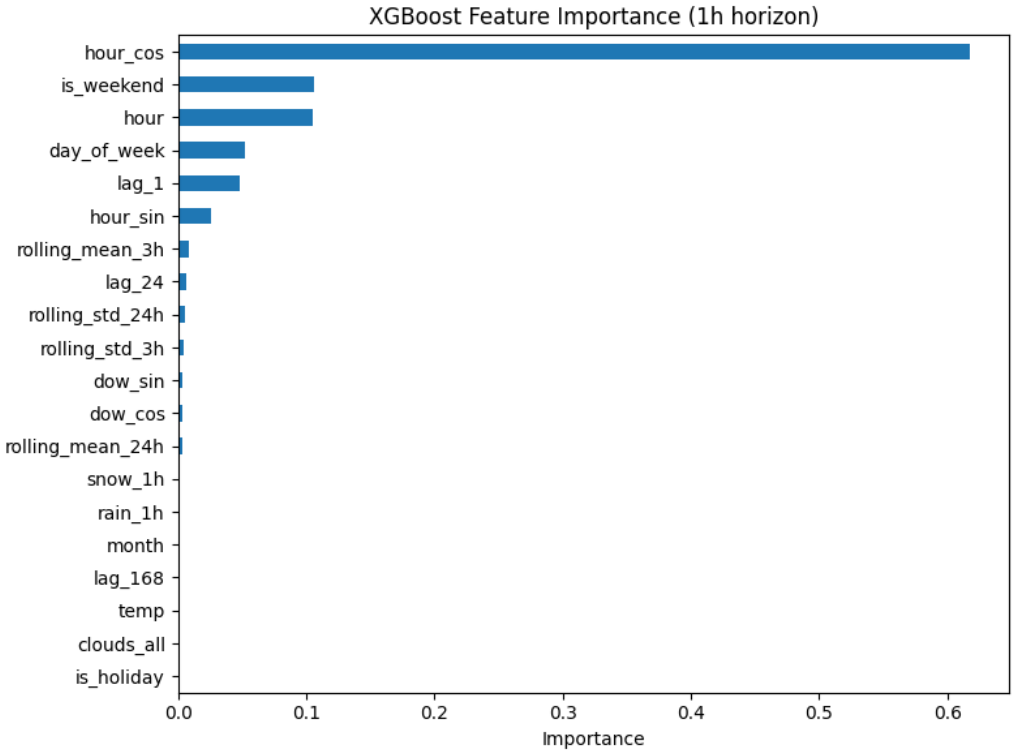

Figure 10: XGBoost Feature Importance for the 1-hour forecasting horizon.

9 Discussion

9.1 Key Findings

The main findings from the experimental evaluation can be summarized as follows:

- **Short-term forecasting:** XGBoost achieved the best performance at 1, 3, and 6 hours ahead.
- **Intermediate forecasting:** LSTM achieved the best performance at the 12-hour horizon, indicating that sequential learning provided an advantage over the tree-based model at this horizon.
- **Long-term forecasting:** The Seasonal Naive model achieved the best performance at 24 hours, demonstrating the strong weekly seasonality present in the dataset.
- **Prophet performance:** Prophet consistently produced higher errors than the other evaluated models, suggesting that the selected trend, seasonal, and external-regressor representation was insufficient to capture the short-term dynamics of the traffic series.
- **Model complexity:** The results demonstrate that increasing model complexity does not necessarily guarantee improved forecasting accuracy. The simple Seasonal Naive baseline remained competitive and achieved the best 24-hour performance.

9.2 Best Model by Horizon

Horizon	Best Model	Reason
1h	XGBoost	Lowest overall error (MASE = 0.5197)
3h	XGBoost	Superior non-linear momentum tracking
6h	XGBoost	Best utilization of recent lag features
12h	LSTM	Best intermediate temporal sequence learning
24h	Seasonal Naive	Lowest error; highest stability at extended limits

Table 5: Best Model Justification by Forecasting Horizon

10 Limitations

- **Data Gaps:** After duplicate removal, comparison with the theoretical continuous hourly timeline revealed 11,976 missing timestamps, representing approximately 22.8% of the reconstructed timeline. This repeatedly breaks continuous feature momentum.
- **Prophet Constraints:** The Prophet implementation primarily models trend, seasonality, and selected external regressors, without the engineered lag and rolling features provided to XGBoost. This may limit its ability to capture the short-term autoregressive behavior present in the traffic series.

- **Long-Horizon ML Decay:** Both XGBoost and LSTM failed to outperform the Seasonal Naive baseline at the 24-hour horizon, indicating that additional temporal representations may be required for longer-term forecasting.
- **Hardware Constraints:** Extensive hyperparameter optimization (such as deeper grid searches for the LSTM network) was constrained by the computational limitations of the older generation Intel Core i5 processor utilized for the project, which experienced processing lag during sequential model training.

11 Conclusion

This project successfully developed and evaluated a multi-step hourly traffic forecasting pipeline using Seasonal Naive, Prophet, XGBoost, and LSTM models.

The experimental results demonstrate that forecasting performance is strongly dependent on the prediction horizon. XGBoost achieved the best performance at the 1-, 3-, and 6-hour horizons, with MASE values of 0.5197, 0.6390, and 0.7280, respectively. At the 12-hour horizon, LSTM achieved the lowest MASE of 0.8105, outperforming XGBoost's 0.8709. At the 24-hour horizon, the Seasonal Naive model achieved the best performance with a MASE of 0.9404.

Prophet consistently underperformed the other models across all evaluated horizons.

Overall, the results show that model selection should depend on the forecasting horizon rather than assuming that a single complex model will always perform best. XGBoost is particularly effective for short-term traffic forecasting, LSTM is competitive for intermediate horizons, and the Seasonal Naive approach remains a strong benchmark for longer-horizon prediction.

12 Future Work

- **Advanced Architectures:** Implementing Attention mechanisms or Transformer models to better handle long-sequence dependencies compared to standard LSTMs.
- **Hyperparameter Optimization:** Utilizing Bayesian optimization libraries like Optuna to tune models individually per forecasting horizon.
- **Recursive Strategy Evaluation:** Comparing the current direct forecasting approach against a recursive feedback loop for 24-hour forecasting.

13 References

1. UCI Machine Learning Repository. *Metro Interstate Traffic Volume Data Set.*

2. Taylor, S. J., & Letham, B. "Forecasting at Scale." *The American Statistician.*

3. Chen, T., & Guestrin, C. "XGBoost: A Scalable Tree Boosting System." *KDD.*

4. Hyndman, R. J., & Koehler, A. B. "Another look at measures of forecast accuracy" (MASE formulation).